Data-Structures & Algorithms

with

Manoj Prabhakaran

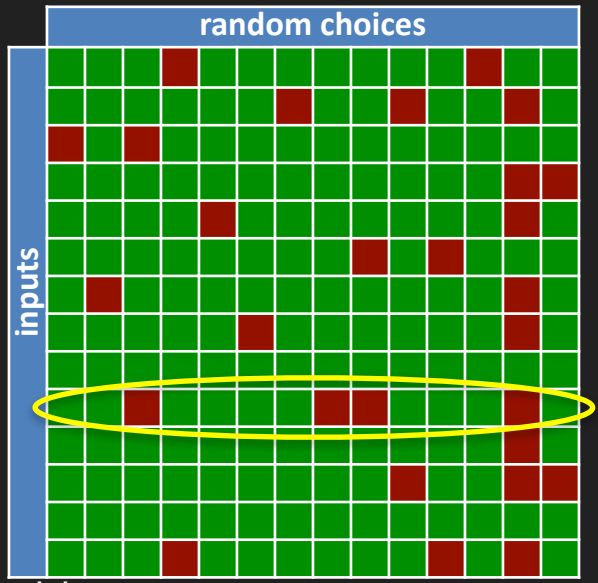
Lecture 38

Introduction to Randomisation

Skip List and Bloom Filter

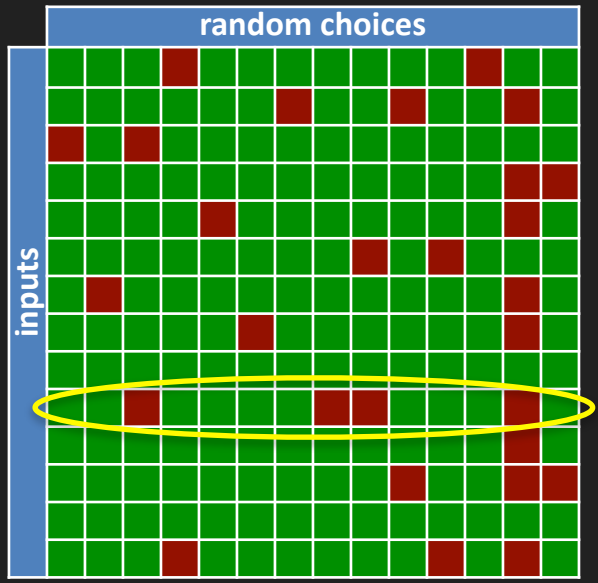
Measuring Costs

- For any given input, the cost (time, space) of a randomised algorithm is not a single number
  - It is a probability distribution over numbers
- Suppose our algorithm uses k random bits to make its random choices
  - Then 2^k possibilities, each with probability $1/2^k$
  - So time taken on a given input can be represented by a vector of 2^k numbers
  - How do we compare two cost vectors? What is a worst-case input now? How do we compare to algorithms (each with their own matrix)?



Measuring Costs

- Instead of the whole distribution (vector) we consider its *statistics* (one, or a few numbers)
- Most important statistic: Expected value, i.e., average of this vector
  - If we run many independent instances of the algorithm on this input, the empirical average will be very close to the expected value with extremely high probability



- Other statistics: Variance (a measure of how much the distribution is spread around the expectation), tail bounds (bound on the probability of seeing values very far from the expectation in a single run), ...

Attendance Break!

Login using CSE LDAP id at

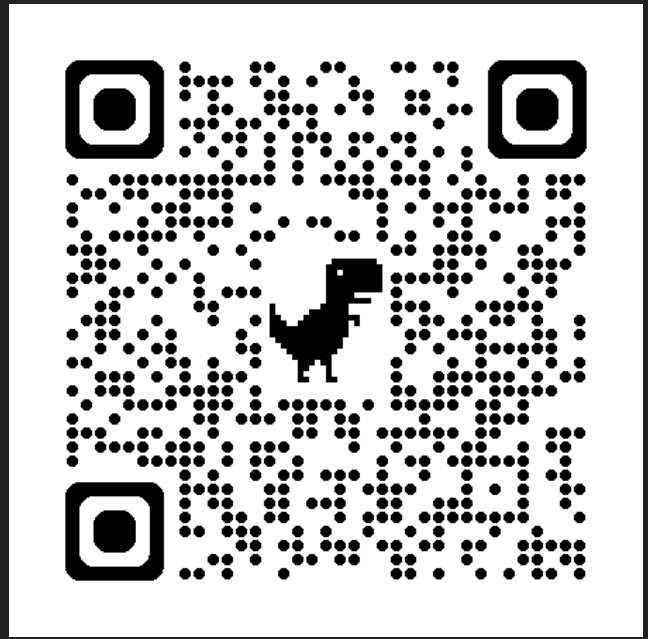
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

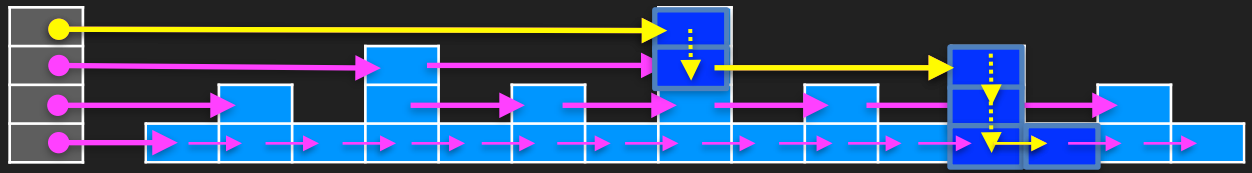
If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Skip List

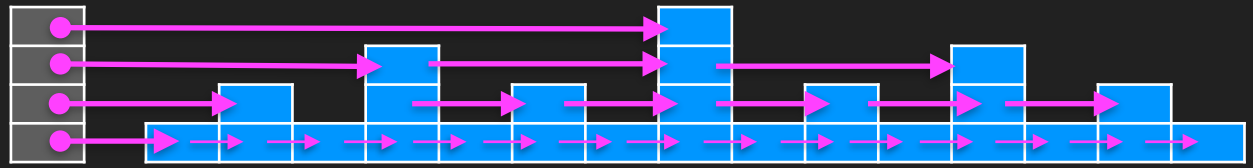
- A simple alternative to Balanced BST
- In a perfectly balanced BST, the tree's "width" at each level doubles
  - Each level partitions the elements into ranges (which get finer as we go down the tree)
- Can mimic these with linked lists!



- Searching in O(log n) time: starting from the top list and left sentinel, linearly search for last node < key, and then move one level down. Continue searching in the next level from the current node.

Skip List

- Problem: How do we handle insertions and deletions?
- If the higher levels contain exactly the middle elements from each range below, inserting or deleting will require the upper lists to be entirely reorganised
- Solution: Each element in a level is promoted to the next level with probability 0.5
  - Ensures that, with high probability, each level has roughly half as many elements as the level below, and they are almost evenly spread. Enough to give O(log n) search (with high probability). Analysis omitted
  - And it solves our problem: Each element's height is independent of the others'!



Bloom Filter

- Another data structure for membership query
- How little space can we use to store n items from a (finite) universe of size N?
  - To distinguish between all $\binom{N}{n}$ sets: $\log \binom{N}{n} = \Theta(n \log(N/n))$ bits
  - With a randomised dictionary mapping each set to a representation, and an ε probability of error, Θ(n log (1/ε)) bits enough (independent of N)!
    - But membership queries and updates are not efficient: need to lookup the whole set in the dictionary
  - Bloom filter: An efficient mechanism with ε probability of false positives (and no false negatives), for search and insertions, using Θ(n log (1/ε)) bits

Bloom Filter

- Initialise an m-bit array A to all 0s (bit array is packed into, say, integers)
- Z be the universe of elements. Pick $h_i : Z \to \{0,\ldots,m-1\}$, for i = 1 to k, from a family of hash functions (this is what is randomised)
- To insert an item $x \in Z$: for each i, set `A[ h_i(x) ] = 1`
- To check if an item $x \in Z$: check if for all i `A[ h_i(x) ] == 1`
  - Clearly, if x was inserted, checking will find it (no false negatives)
  - If x was not inserted, but insertion of other elements turned all k bits checked to 1, we will have a false positive

Analysis omitted

  - False positive probability $\leq \varepsilon$ with $k = \Theta(\log(1/\varepsilon))$, and $m = \Theta(nk)$. Space: O(k) bits per item. Worst-case time: O(k) per insertion/search.

And That's a Wrap!

